ADVANCED PYTEST CONCEPTS (WITH CODE)

---

1. How to Call Parametrized Tests Properly

Concept

`@pytest.mark.parametrize` allows **same test logic** to run with **multiple inputs**.

Instead of writing many test functions, we write **one test with multiple data sets**.

Single Parameter

```
import pytest

@pytest.mark.parametrize("x", [1, 2, 3, 4])
def test_square_positive(x):
    assert x * x > 0
```

What happens:

- Pytest runs this test **4 times**
- Each time with a different value of `x`

Execution internally:

```
test_square_positive(1)
test_square_positive(2)
test_square_positive(3)
test_square_positive(4)
```

Multiple Parameters

```
@pytest.mark.parametrize(
    "a, b, expected",
    [
```

```
        (2, 3, 5),
        (5, 5, 10),
        (10, 0, 10)
    ]
)
def test_addition(a, b, expected):
    assert a + b == expected
```

Exam angle:

- Reduces duplicate code
- Improves test coverage

---

2. How to Run Test Case and Test Suite in Pytest

Pytest **does not explicitly define test suites** like unittest.
Instead, **test discovery rules** are used.

Running a Single Test File

pytest test_math.py

Runs:

- All test functions in that file

Running a Single Test Function

pytest test_math.py::test_addition

Runs:

- Only `test_addition`

Running Tests from a Class

pytest test_math.py::TestCalculator

Runs:

- All test methods inside `TestCalculator`

Running a Single Test Method in a Class

pytest test_math.py::TestCalculator::test_add

Running All Tests (Test Suite Equivalent)

pytest

This automatically:

- Finds all files starting with `test_`
- Runs all test cases

Exam Note:
In Pytest, **test suite = collection of discovered tests**, not a manually created object.

---

3. Running Tests Using Markers (Suite-like Behavior)

Marking Tests

```
import pytest

@pytest.mark.slow
def test_large_computation():
    assert 1000000 * 1000000 > 0
```

Run Only Slow Tests

pytest -m slow

Run All Except Slow Tests

pytest -m "not slow"

This acts like **logical test grouping**, similar to test suites.

---

4. What If a Fixture Sets More Than One Variable

Option 1: Return a Tuple

```
import pytest

@pytest.fixture
def setup_data():
    a = 10
    b = 20
    return a, b
```

Usage:

```
def test_sum(setup_data):
    a, b = setup_data
    assert a + b == 30
```

Option 2: Return a Dictionary (Most Common)

```
@pytest.fixture
def setup_data():
    return {
        "x": 5,
        "y": 10,
        "z": 15
    }
```

Usage:

```
def test_total(setup_data):
    assert setup_data["x"] + setup_data["y"] + setup_data["z"] == 30
```

Option 3: Yield Multiple Variables (Setup + Teardown)

```
@pytest.fixture
def setup_resources():
    x = 10
    y = 20
    print("Setup resources")
    yield x, y
    print("Teardown resources")
```

Usage:

```
def test_resources(setup_resources):
    x, y = setup_resources
    assert x * y == 200
```

---

5. Using Fixtures Automatically (Without Passing as Parameter)

Concept

Sometimes we want a fixture to run **automatically** for tests.

Using `usefixtures`

```
@pytest.fixture
def init_db():
    print("DB initialized")

@pytest.mark.usefixtures("init_db")
def test_insert():
    assert True
```

Fixture runs even though:

- It is not passed as an argument

Autouse Fixtures

```
@pytest.fixture(autouse=True)
def setup_env():
    print("Environment setup")
```

Runs:

- Before **every test automatically**

Exam note:

- Use carefully
- Can make debugging harder

---

6. Fixture Scope (Very Important for Exam)

Fixture Scope Types

Scope	Runs
function	Before every test
class	Once per test class
module	Once per file
session	Once for whole test run

Example: Class Scope

```
@pytest.fixture(scope="class")
def shared_data():
    return 100
```

Usage:

```
class TestExample:
```

```
    def test_one(self, shared_data):
        assert shared_data == 100

    def test_two(self, shared_data):
        assert shared_data > 0
```

Fixture runs:

- Once for the entire class

---

7. Combining Fixtures and Parametrization

```
@pytest.fixture
def base():
    return 10

@pytest.mark.parametrize("x", [1, 2, 3])
def test_multiply(base, x):
    assert base * x > 0
```

Flow:

- Fixture provides base value
- Parametrize supplies test data
